Department of Computer Science and Engineering

1. Subject Code: **CPT-301** Course Title: **Programming Methodologies**

2. **Expected Effort:** Total: 150 Hours Pacing: 10 Hours / Week Duration: 15 Weeks

3. **Prerequisites**
   - Basic Programming Fundamentals (variables, loops, conditionals), High School Mathematics, Logical Problem Solving.

4. **Course Structure**
   5 Units × 30 Hours (lectures, source-code reading, laboratory, PoW)

1. Course Description

This course builds algorithmic reasoning from first principles to interview-grade execution. Students move from mechanical tracing of memory state, through the canonical linear and non-linear data structures, into the optimisation paradigms (greedy, backtracking, graphs, dynamic programming) that underpin every later course in the programme. Every unit is anchored to a timed Proof-of-Work in which correctness, time complexity and space complexity are all graded.

2. Course Outcomes

After successful completion of the course, students will be able to:

CO1. Analyse problem statements and apply optimal array, string, and pointer-based traversal techniques.
CO2. Design and manipulate linear and non-linear core data structures including Linked Lists, Trees, and Heaps.
CO3. Formulate optimisation strategies using Greedy algorithms, Interval reasoning, and Backtracking paradigms.
CO4. Develop highly efficient solutions for complex relationships using Graph Traversals, Union-Find, and Dynamic Programming.
CO5. Synthesise advanced systems of patterns (Tries, Monotonic Stacks, Custom DS Design) while demonstrating elite technical communication and mock interview execution.

3. Detailed Syllabus

Unit	Contents	Hrs
1	**Mechanical Reasoning, Arrays & Early Patterns** *Module A: Reasoning Foundations* Mechanical reasoning and memory state changes. Brute force versus optimised approaches ($O(n^2) \rightarrow O(n)$). Asymptotic notation, amortised intuition, loop invariants, and correctness arguments. *Module B: Arrays, Strings & Hashing* In-place array manipulation, rotation and partitioning. String processing and character frequency models. Hash Maps and Hash Sets, collision intuition. Prefix Sums and difference arrays. *Module C: Early Pattern Recognition* Two Pointers (opposite-end and same-direction). Fixed and Variable Sliding Windows. Binary Search invariants: first/last occurrence, boundary search, search over a monotonic predicate. **Hands-on Laboratory & Proof-of-Work (PoW)** • Implement array manipulations transitioning from brute force to $O(n)$ using Hash Maps and Two Pointers. • Solve canonical sliding-window and binary-search problems under strict time constraints. • **PoW:** Build a personal `algo-lib` repository with tested primitives for prefix sums, two pointers and binary-search templates.	30
2	**Data Structure Intuition: Recursion, Lists, Trees & Heaps** *Module A: Recursion, Stacks & Queues* Recursion as delayed processing: call trees, base cases, trace diagrams, recursion-to-iteration conversion. Stack operations for state validation and undo semantics. Queue mechanics and arrival ordering. *Module B: Linked Structures* Linked List pointer rewiring, dummy nodes, fast/slow pointers, cycle detection, in-place reversal, merge patterns. *Module C: Trees & Heaps* Binary Tree traversals (DFS pre/in/post-order, BFS level order) and structural invariants. Binary Search Trees and the ordering property. Min/Max Heaps, heapify, and top-k selection with Priority Queues. **Hands-on Laboratory & Proof-of-Work (PoW)** • Build custom Linked List rewiring scripts with dummy-node and fast/slow-pointer variants. • Implement pre-order, in-order and post-order traversals iteratively and recursively. • **PoW:** Construct a system that dynamically maintains the K-th largest element of a live stream using Priority Queues.	30

Unit	Contents (continued)	Hrs
3	**Algorithmic Optimisation I: Greedy, Intervals & Backtracking** *Module A: Greedy Foundations & Intervals* Local choice safety and exchange arguments. Reachability problems. Interval mathematics: sorting strategies, overlap merging, insertion, and line-sweep counting. *Module B: Binary Search on the Answer* Feasibility predicates over monotonic answer spaces. Minimising the maximum and maximising the minimum. Precision search on real-valued domains. *Module C: Backtracking & Bit Manipulation* The choose-explore-unchoose paradigm. Permutation and subset generation. Recursion-tree pruning and constraint propagation. Bit manipulation: XOR cancellation, parity checks, subset masks. **Hands-on Laboratory & Proof-of-Work (PoW)** • Develop a meeting-room scheduling algorithm using interval merging and line sweep. • Build an exhaustive backtracking solver for combinatorial problems (subsets, permutations, path finding, N-Queens). • **PoW:** Solve a capacity-allocation problem using binary search on the answer with a documented feasibility proof.	30
4	**Algorithmic Optimisation II: Graph Theory & Dynamic Programming** *Module A: Graph Modelling & Traversal* Adjacency lists versus matrices, implicit graphs, connected components. DFS state graphs and cycle detection. Multi-source BFS, Topological Sort, and unweighted/weighted shortest paths. *Module B: Disjoint Set Union* Union-Find with path compression and union by rank/size. Connectivity queries, Kruskal preparation, and offline query processing. *Module C: Dynamic Programming Foundations* Overlapping subproblems and optimal substructure. Memoisation versus tabulation. 1D transitions, subset sum (boolean DP), grid DP, and 2D string DP (LCS, edit distance). **Hands-on Laboratory & Proof-of-Work (PoW)** • Model real-world networks as graphs to detect cycles and compute shortest paths. • Optimise recursive brute-force algorithms into bottom-up DP arrays and reduce space to $O(n)$ rolling rows. • **PoW:** Build a dependency-resolution tool using topological sorting with cycle diagnostics.	30

Unit	Contents (continued)	Hrs
5	**Advanced Pattern Composition & Elite Technical Execution** *Module A: Monotonic Structures & Tries* Monotonic Stack for next-greater/smaller element and range invariants. Monotonic Deque for sliding-window extremes. Trie structures: prefix trees, wildcard branching, and word dictionaries. *Module B: Data Structure Design APIs* Composing Hash Maps with Lists and Heaps to satisfy strict API contracts: LRU Cache, MinStack, Randomised Set, Time-Based Key-Value store. *Module C: Interview Protocol & Communication* Strict technical interview protocol. Verbal articulation of trade-offs, clarifying ambiguity, complexity justification, and structured debugging under stress. **Hands-on Laboratory & Proof-of-Work (PoW)** • **PoW:** Design and implement a fully functional LRU Cache with $O(1)$ get and put. • **PoW:** Participate in timed, company-specific (Google/Meta/Amazon style) mock assessments demanding precise code and verbal defence. • Maintain a written trade-off log for every capstone problem attempted.	30
	Total Instructional Effort	**150**

4. Recommended Resources

- Cormen, Leiserson, Rivest & Stein, *Introduction to Algorithms*, 4th ed.
- Skiena, *The Algorithm Design Manual*, 3rd ed.
- Sedgewick & Wayne, *Algorithms*, 4th ed.
- Curated practice ladders on LeetCode and Codeforces, mapped unit-wise by the instructor.
- Language reference: C++ STL containers/algorithms or Python `collections`, `heapq`, `bisect`.